

Personal Research Assistant — Interview Guide

Everything you need to talk about this project confidently: the pitch, the architecture, the parts that are genuinely hard, the questions you'll be asked, and the honest answers to the uncomfortable ones.

How to use this. Read §1 and §9 the night before. Skim §5 — those six points are what actually separate this project from a tutorial. Keep §7 open in a second window if it's a remote interview.

1. The pitch

30 seconds

"I built a Personal Research Assistant — a multi-agent system on LangGraph where a planner breaks a question into subtasks, a human approves the plan, and then a supervisor routes each subtask to one of three specialist agents, with a critic reviewing every answer before it's accepted. It's a full product, not a notebook: FastAPI backend, React frontend, Postgres with pgvector for document search, bearer-token auth, per-user document isolation, rate limiting, CI, and it's deployed. I also kept the 25 incremental stages I built it through, so you can diff any stage against the next and see exactly which concept was added."

2 minutes — add the shape of it

"The core insight is that a research question isn't one LLM call, it's a pipeline. So the graph has layers.

The outer graph plans: it takes your question, breaks it into two or three subtasks, and then **stops** — it calls LangGraph's `interrupt()` and waits for a human to approve the plan. That pause is persisted to Postgres, so it genuinely spans two separate HTTP requests, and survives a server restart.

Once approved, an executor loop runs each subtask. For each one it invokes a whole second graph: a supervisor reads the subtask and routes it to a Research agent (web search), a Knowledge agent (semantic search over your uploaded documents), or an Analysis agent (arithmetic). Then a critic node reviews that answer and either passes it or sends it back once with feedback.

After each subtask the system reflects — it can notice a gap and add a follow-up subtask it discovers it needs. But it can never decide to *stop*; termination is a pure function with a hard step budget. That split is deliberate and it's the part I'd most want to talk about."

2. Architecture

Request lifecycle



`POST /reject` resumes the same interrupt with `"n"`, and `route_after_approval` sends it straight to `END` — no research runs.

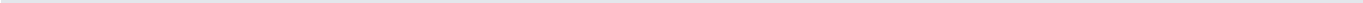
Two graphs, composed

	Outer (<code>app/graphs/planner.py</code>)	Inner (<code>app/graphs/specialist.py</code>)
Job	plan → approve → loop → synthesize	route → answer → review
State	<code>PlannerState</code> (TypedDict)	<code>CriticState</code>
Checkpointed	Yes — <code>AsyncPostgresSaver</code>	No — fresh per subtask
Human in it	Yes, one <code>interrupt()</code>	No

The inner graph is invoked as a plain callable inside the outer graph's `react_step` node. **This is the single most reusable idea in the project:** a compiled LangGraph graph is just a runnable, so a whole pipeline can be a node inside another pipeline. Say this out loud in the interview — it shows you understand composition, not just wiring.

Repo layout

app/	FastAPI + LangGraph backend	- 37 files, ~3,200 lines
frontend/	React + TypeScript SPA	- 33 .ts/.tsx files
stages/	25 self-contained learning stages	
tests/	pytest suite	- 20 files, ~2,300 lines, 132 tests
docs/	specs, plans, PROGRESS log	



3. Stack, and why each piece

Layer	Choice	The reason to give
Orchestration	LangGraph	The flow has cycles, a conditional branch, and a mid-graph pause. A DAG or a plain chain can't express any of those.
LLM	gpt-4o-mini	Cheap enough to run a 3-subtask pipeline (10+ calls) per question. Quality is sufficient because each agent has a narrow job.
Embeddings	text-embedding-3-small , 1536-dim	Matches the <code>vector(1536)</code> column; small and cheap for chunk-level retrieval.
API	FastAPI	Async-native (the whole graph is <code>async</code>), and Pydantic response models give a typed OpenAPI contract the frontend and the parity test both consume.
Persistence	Postgres + pgvector	One store for both the checkpoints and the document vectors. No separate vector DB to operate.
Checkpointing	AsyncPostgresSaver	This is what makes the human-approval pause durable rather than in-memory.
Frontend	React + TypeScript + Vite	Typed against the same response shapes; Vitest for the pure logic.
Deploy	Neon → Render → Vercel	Managed Postgres, container backend, static frontend.

Temperature is split on purpose. Seven LLM clients: `plan`, `reflect`, `synthesize`, and `greet` share one client left at the API default because they *write*; the supervisor, critic, three specialists, and intent classifier are each pinned to `temperature=0.0` because they *decide*. A classification that varies between runs is pure downside — the same subtask routing to a different specialist on two identical runs would make the system impossible to reason about.

4. The 25-stage archive

Not filler — this is a differentiator, and interviewers respond to it.

Stage 1 is a `StateGraph` with one node and a `MemorySaver`. Stage 25 is a deployed multi-agent system with a React UI. Every stage in between is **self-contained and runnable**, and adds exactly one concept.

The design rule is the interesting part: **later stages deliberately duplicate setup code from earlier ones instead of importing a shared module**. That looks like a mistake until you know the goal — each folder must be readable top-to-bottom on its own, so you can `diff` stage N against stage N+1 and see precisely which lines the new concept cost. A `common/` module would hide that diff. Extracting shared code would optimize for the wrong thing.

Milestones worth naming: **2** tool calling, **3** RAG, **6** custom state + hand-written conditional edge, **7** human-in-the-loop `interrupt()`, **13** supervisor routing, **14** critic, **17** the capstone composition, **18** Postgres persistence, **19** FastAPI, **21** pgvector, **23** per-user isolation, **24** security guardrails, **25** React UI.

`app/` is a production-style **port** of Stage 25 — the stages stay frozen and runnable; `app/` is where the engineering (async, pooling, auth, tests) went.

5. What's actually excellent — lead with these

These six are what make this more than a tutorial. Each one is a real engineering decision with a defensible reason.

5.1 Human-in-the-loop that survives a process restart

`human_approval` calls `interrupt()`. The graph state — including the pending interrupt — is written to Postgres by the checkpointer. `POST /chat` returns `202`-style status `awaiting_approval` and the process is free. Minutes later (or after a deploy), `POST /approve` calls `graph.ainvoke(Command(resume="y"))` on the same `thread_id` and the graph continues from exactly where it stopped.

Why it's hard: a pause inside a running graph is easy in a REPL and hard across a stateless HTTP API. It requires the checkpointer to be load-bearing, `graph.aget_state()` to validate the resume is legal, and a per-`thread_id` lock so a concurrent `/chat` can't slip a new checkpoint in between the check and the resume.

Say: "The approval isn't a UI confirmation dialog — it's a durable pause in the graph itself. The state lives in Postgres between the two requests."

5.2 The LLM reflects, but never decides when to stop

`decide_next_action()` is a **pure function with no LLM call**:

```
if state["step_count"] >= MAX_REACT_STEPS:      # budget checked FIRST
    return ReactDecision(action="finish", reason="step_budget_exhausted")
if not state["agenda"]:
    return ReactDecision(action="finish", reason="agenda_empty")
return ReactDecision(action="research", subtask=state["agenda"][0], ...)
```

`reflect()` — which *does* call the LLM — may only **append** to the agenda. It cannot end the loop and it cannot skip approved work. The budget is checked *before* the agenda so a full agenda can never outvote the ceiling.

There is a test that swaps in an LLM client which raises `AssertionError` on any call, purely to prove `decide_next_action` never reaches it.

Say: "An agent that chooses its own exit condition can't be reasoned about. So the model reasons about *results*; control flow stays deterministic and auditable."

5.3 The approval gate isn't decorative

Because `reflect()` can add subtasks *after* a human approved a plan, the trace would be a lie if it couldn't tell the two apart. So every trace entry carries `origin: "approved" | "agent"` — approved means it was in the plan the human actually saw.

This is a small field that shows a specific kind of thinking: *I added a capability, and I noticed it silently undermined an existing guarantee, so I made the guarantee observable again*. Interviewers notice that.

5.4 Prompt-injection defense in depth

Three independent layers, each doing something the others can't:

1. **The model cannot name a victim.** `search_uploaded_documents` receives `user_id` via `langgraph.prebuilt.InjectedState` — it comes from graph state, is invisible to the LLM, and isn't part of the tool schema. No prompt can talk the model into supplying someone else's id, because the model has no parameter to put it in.
2. **Retrieved text is framed as data.** Every non-empty result is wrapped in an explicit untrusted-content envelope telling the model that what follows is reference text, not instructions.
3. **The output is checked, not the input.** A deterministic, non-LLM guard looks for a ≥ 40 -character verbatim span of the system prompt in the final answer and replaces it. Checking the *output* for a leak instead of the *input* for an attempt means it needs no maintenance as new jailbreak phrasings are invented.

Plus: a foreign `document_id` returns **404, not 403**, so the endpoint can't be used to probe which ids exist.

Say: "Layer 1 makes the attack impossible rather than difficult. That's why it's first."

5.5 Tests that encode properties, not coverage

132 tests. The design constraint: **the normal run never calls the real OpenAI API** — it costs money and makes the suite non-deterministic. Only 8 e2e tests hit the real API, gated behind a fixture that skips whenever `OPENAI_API_KEY` is unset. CI passes a deliberately-unusable placeholder key so `import app` succeeds while the gate still holds.

Two tests are worth describing by name:

- `test_schema_parity.py` — loads the *original* Stage 25 backend and the new `app/` package, generates both OpenAPI schemas, and asserts no route was removed, no model was removed, no property was removed, and **no enum was narrowed**. Widening is allowed; narrowing fails the build. It's a machine-checked backward-compatibility contract between a refactor and the thing it replaced.
- `test_budget_is_checked_before_the_agenda` — asserts the *ordering* of two conditions, because that ordering is what makes the safety ceiling real.

5.6 Configuration as a startup contract

`validate_for_startup()` refuses to boot rather than failing one request at a time. In prod it rejects: an unset `DATABASE_URL` that silently fell back to the local Docker database, a connection string with `sslmode=disable` (document text and embeddings would cross the network in the clear), an unset `AUTH_SECRET_KEY` (an ephemeral key would log every user out on restart and differ between replicas), and an empty CORS allow-list.

It collects **every** problem and reports them together — an operator fixing a misconfigured deploy should see the whole list once, not rediscover it one restart at a time.

6. Deep-dive cheat sheets

LangGraph concepts you should be able to define cold

Term	One-line answer
<code>StateGraph</code>	A graph whose nodes read a shared typed state and return a <i>partial</i> update, which is merged in.
Node	A function <code>state -> dict</code> . Returning only changed keys is why <code>plan()</code> can reset fields without touching <code>user_name</code> .
Conditional edge	A function returning the <i>name</i> of the next node — how routing decisions become graph structure.
Checkpoint	Persists state per <code>thread_id</code> after every super-step. Here <code>AsyncPostgresSaver</code> , which is what makes the pause durable.
<code>thread_id</code>	The conversation key, passed as <code>config={"configurable": {"thread_id": ...}}</code> . The checkpointer merges history, so callers only ever send the new message.
<code>interrupt()</code>	Suspends the graph mid-node and surfaces a value to the caller.
<code>Command(resume=v)</code>	Restarts a suspended graph, with <code>v</code> becoming <code>interrupt()</code> 's return value.
<code>InjectedState</code>	Marks a tool argument as supplied from graph state, not by the model. The security primitive in §5.4.
Subgraph	A compiled graph used as a node. Here: the whole supervisor+critic pipeline inside one planner node.

Data layer

- `documents` — id, filename, file_type, size, chunk_count, `user_id`, uploaded_at
- `document_chunks` — id, document_id (FK, `ON DELETE CASCADE`), chunk_index, content, `embedding vector(1536)`
- Retrieval: `ORDER BY dc.embedding <=> %s` (pgvector cosine distance), always with `WHERE d.user_id = %s`
- Chunking: 400 chars, 50 overlap. Agent tool retrieves `k=3`.
- Schema is created idempotently at startup; the `embedding` column arrived as a real `ALTER TABLE ... ADD COLUMN IF NOT EXISTS` migration in Stage 21.
- pgvector is registered per-connection via the pool's `configure` hook — with a pool, a one-time `register_vector()` leaves later connections not knowing the type, and search fails *unpredictably*.

Reliability details worth mentioning

- A crashing specialist doesn't kill the run.** `react_step` catches, records a generic placeholder and a `status: "failed"` trace entry, consumes the subtask and spends the step — so the loop can't re-dispatch the same failing subtask until the budget drains. Before that fix, one flaky web search discarded a three-subtask run's completed work.
- Exception text never reaches the caller** — it can carry a connection string or a prompt fragment. Logged server-side, generic message out.

- **Every LLM call has a timeout** (60s) and bounded retries (2). Without one, a hung upstream call held both the request and its per-thread lock.
 - **synthesize** **pairs against the trace, not the plan** — a follow-up subtask would otherwise mis-pair every answer after it.
 - **A failed subtask is named as such in the synthesis prompt**, so the final answer says what it couldn't cover instead of reporting the failure as a finding.
-

7. Hard questions, and how to answer them

"Why LangGraph instead of just calling the API in a loop?"

Three things I'd have had to build myself: cyclic control flow with a checkpoint after every step, a durable mid-graph pause for human approval, and typed state merging across nodes. The checkpoint is the one I'd genuinely not want to write — durable resume across a stateless HTTP API is the hard part of this whole project.

"How do you stop the agent looping forever?"

Termination is a pure function, not an LLM decision — §5.2. Hard budget of 6 steps, checked before the agenda, and every path back into the executor goes through the same routing function, so there's no way around the ceiling.

"How is multi-tenancy enforced?"

Two halves. Filtering: `WHERE d.user_id = %s` on both retrieval paths. Authentication: the `user_id` comes from an HMAC-SHA256-signed bearer token the server issued, never from the request body. Before I added auth the filtering was already correct, but it was *arithmetic, not a boundary* — anyone who guessed a `user_id` could read those documents. And inside the agent, the tool's `user_id` is injected from graph state so the model can't supply one.

"Why is the critic an LLM? Isn't that circular?"

Partly, yes — and I'd call that the weakest link. It catches obvious non-answers and off-topic responses, which is worth the one retry it's budgeted. It is not a correctness oracle, and I wouldn't claim it is. A real eval harness with fixed cases and scored outputs is the honest next step.

"Why doesn't it stream?"

There's no streaming transport — `POST /approve` is one blocking call that runs the entire loop before returning. That was a deliberate scope decision: a live feed means real-time infrastructure, and the UI is honest about it ("Researching... this can take a little while") rather than faking progress. With `astream_events` and SSE it's a well-defined next piece of work.

"What's the cost/latency profile?"

A 3-subtask question is roughly 10–15 `gpt-4o-mini` calls: 1 classify, 1 plan, then per subtask a supervisor route + a specialist ReAct turn + a critic review (+ possible retry), then reflect and synthesize. Tens of seconds. That's what buys the approval gate — the user sees the plan before paying for any of it.

"Walk me through adding a fourth specialist."

Three edits and no new concepts, which is the point of the design. Write the agent module with its tool; add the node and one dispatch entry to the inner graph; widen the supervisor's routing `Literal`. The critic needed zero changes when I added the third one — the design was already right.

"What would you do differently?"

Answer honestly, in this order — see §8.

8. Known limitations — name them before they're found

Volunteering these makes every other claim more credible.

Limitation	The honest framing
Auth is a bootstrap credential, not an IdP	One shared signup secret; no user table, no passwords, no revocation. It stops a stranger reading another user's documents. It does not stop whoever holds the secret from minting a token for any user_id. I ranked those two and closed the one that mattered with real documents in the database.
Rate limiting is an in-process dict	Correct for a single instance; useless across replicas. Redis is the change, and it's a swap of one module.
No ANN index on the vectors	Every search is an exact cosine scan over the user's chunks. Right at this data volume; an <code>ivfflat</code> / <code>hnsw</code> index is the first thing to add when it isn't.
The critic is an LLM judging an LLM	See §7. Useful heuristic, not a correctness guarantee.
No streaming	See §7.
Postgres also holds the checkpoints	One store to operate, but checkpoint writes and vector search now share a connection pool. Fine here; would separate under load.

9. Numbers to have ready

Learning stages	25, each self-contained and runnable
Backend	37 Python files, ~3,200 lines
Tests	132 , in 20 files (~2,300 lines)
Network-free run	101 pass, 23 skip (need Postgres)
Real-API tests	8, gated behind a fixture
Frontend	33 <code>.ts</code> / <code>.tsx</code> files, 10 Vitest tests
Agents	1 planner + 1 supervisor + 3 specialists + 1 critic
LLM clients	7, temperature split between <i>deciding</i> (0.0) and <i>writing</i> (default)
Step budget	<code>MAX_REACT_STEPS = 6</code>
Retry budget	<code>MAX_RETRIES = 1</code> (2 specialist attempts total)
Embeddings	1536-dim, cosine distance (<code><=></code>)
Chunking	400 chars, 50 overlap; agent retrieves k=3
Upload caps	20 MB, 500 PDF pages, 200 MB unzipped DOCX, 30 s extraction timeout
Input caps	4,000 chars on the question, 100 KB JSON body
Rate limits	chat 10/min per user, 30/min per IP; auth 10/min per IP
Token TTL	12 hours, HMAC-SHA256, stdlib only
Dependencies	Fully pinned including transitive — 90+ lines, derived not <code>pip freeze</code> d
CI	GitHub Actions, Python 3.13 + Node 22, backend and frontend jobs

The dependency-pinning story is a good 30-second answer on its own: the file was 20 bare package names until CI proved that wasn't reproducible. A developer's virtualenv held `openai 3.3.1`; a clean install resolved to a newer SDK that raises at `ChatOpenAI` *construction* rather than at first call — so the suite passed locally and failed on a clean checkout. `openai` is a *transitive* dependency, so pinning only direct dependencies would never have caught it.

10. Demo script (5 minutes)

1. **Show the empty UI.** Point out the three panes: documents, chat, trace.
 2. **Type "hi, I'm <your name>".** It replies warmly and asks what you need — *no plan, no approval panel*.
Explain: an intent classifier routes small talk past the whole planner. One temperature-0 structured call that returns both the branch and the name; anything ambiguous falls back to research, because a question mistaken for small talk is worse than the reverse.
 3. **Upload a PDF.** Note: validated, extracted, chunked, embedded, and stored against your user_id.
 4. **Ask a question about it.** The plan appears — **stop here**. This is the moment to explain that the graph is paused in Postgres, not in memory, and that you could restart the server right now and still approve it.
 5. **Approve.** While it runs, explain supervisor → specialist → critic.
 6. **Open the trace panel.** Per subtask: which specialist, which tools, the critic's verdict, retry count, and whether it was human-approved or added by the agent mid-run.
 7. **Reject one** to show the gate is real — it goes straight to END and no research runs.
-

11. Framing lines that land

- *"Each agent has a narrow job, which is why a small model is enough."*
- *"The model reasons about results; control flow stays deterministic."*
- *"The approval gate would be decorative if the trace couldn't tell approved subtasks from agent-added ones."*
- *"Filtering by user_id was arithmetic, not a boundary, until the id came from a signed token."*
- *"The stages duplicate code on purpose — the diff between two stages is the lesson, and an abstraction would hide it."*
- *"I check the output for a leak rather than the input for an attempt."*

12. Things not to claim

Precision is worth more than superlatives, and one overstatement makes an interviewer discount everything else.

- Don't say "production-grade" — say *production-shaped*: real auth, CI, migrations, guardrails, deployed; single-instance, no IdP, no streaming.
- Don't say the critic guarantees correctness. It doesn't.
- Don't say it's "fully tested" — say 132 tests covering routing, state transitions, safety properties, and the HTTP contract, with the LLM stubbed.
- Don't call the temperature-0 pinning "deterministic". Say *much more consistent* — identical prompts can still diverge upstream from batching and floating-point ordering.
- Don't imply the 25 stages are a framework. They're a learning archive, and that's a better story anyway.